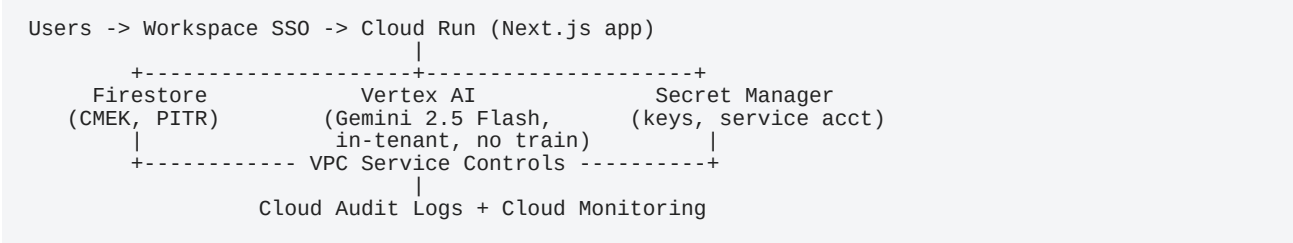
EOS Platform — Tech Stack: What & Why

A walkthrough of every layer of the system, what it does, and why it was chosen. The guiding principle is **one cloud, one perimeter**: the application, its data, and its AI all run inside the client's own Google Cloud (GCP) organization. Nothing of substance leaves the tenant.

The shape of it



Application layer

Next.js 16 (React 19)

What: The web application framework — renders the pages, runs the server-side logic, handles routing. Uses the modern App Router.

Why: It's the industry-standard framework for production React apps, with server-side rendering for fast loads and a server-action model that keeps sensitive logic (and secrets) on the server, never shipped to the browser. Large talent pool, long-term support, and it deploys cleanly to GCP's Cloud Run.

Tailwind CSS v4

What: The styling system used to build the UI, already themed to High Plains Bank's brand colors and typography.

Why: Fast, consistent styling with a small shipped footprint. Keeps the look on-brand without a heavy design-system dependency.

TypeScript + Zod

What: TypeScript adds type-safety across the codebase; Zod validates data at the boundaries (incoming requests, AI responses).

Why: Catches whole categories of bugs before they ship and guarantees that data entering the system is well-formed — important when an AI is proposing structured changes.

Hosting & runtime

Cloud Run (production target — replaces the Vercel demo)

What: Google's managed container platform. The Next.js app runs here.

Why: It puts the web tier in the *same* GCP boundary as the data and the AI, so there's no third-party host to add to the bank's vendor-risk review. It autoscales (including to zero), is usage-priced, and

carries Google's enterprise SLAs. The demo runs on Vercel for speed; production moves here so everything sits in one tenant.

Cloud Build + Artifact Registry

What: The pipeline that builds and ships new versions of the app.

Why: Reproducible, auditable deployments — every release is traceable, which matters in a regulated environment.

Data layer

Firestore (Native mode)

What: The database. Stores all EOS data — teams, rocks (quarterly goals), to-dos, issues, the scorecard, and meeting records.

Why: A fully-managed Google document database with real-time updates (the meeting and scorecard views stay live without page refreshes), automatic scaling, and strong per-document access control. It already lives in GCP, so it's a natural anchor for the "everything in your cloud" architecture.

Access model: Every record is scoped to a **team**. Security rules enforce that a user can only read or write data for teams they belong to — a non-leadership employee cannot see the leadership team's rocks.

Cloud KMS / CMEK

What: Customer-Managed Encryption Keys layered over Firestore's default at-rest encryption.

Why: Lets the bank hold and control the encryption keys for its own data — a common requirement for sensitive financial information and a strong answer to "what if there's a breach."

Point-in-Time Recovery + GCS exports

What: Continuous backup of the database plus scheduled exports to Cloud Storage.

Why: Recover from accidental deletion or corruption to any recent moment, with documented recovery objectives. Standard disaster-recovery hygiene.

Identity & access

Firebase Auth / Identity Platform — Google Workspace SSO

What: Sign-in. Users log in with their existing Google Workspace account; sessions are short-lived, HttpOnly cookies.

Why: No new passwords to manage — the bank's existing Workspace identity *is* the login. Sign-in is **restricted to the bank's domain**, so only `@theirbank.com` accounts can ever authenticate. Supports adding SAML/OIDC (e.g. Okta) later if their identity stack changes.

IAM + Org Policies

What: Google's access-control framework governing who can administer the system and its infrastructure.

Why: Enforces least-privilege. Administrative roles are split (managing users vs. reading content), so

no single role can see everything by default.

AI layer

Gemini 2.5 Flash on Vertex AI

What: Powers the voice/text capture feature — speak or type "close the hiring rock and add a to-do for Dana," and the system proposes the structured changes for a human to confirm.

Why Vertex specifically — this is the key data-governance decision: Running Gemini through **Vertex AI inside the client's GCP project** means

- customer prompts and responses are **not used to train Google's models**,
- inference stays **in-region and in-tenant**, covered by Google's enterprise data-processing terms,
- and it sits behind the same network controls and audit logging as everything else.

By design the feature only ever sends **summaries of the current team's items** (titles, statuses, owner names) — never the whole database — and **nothing is applied automatically**; every AI suggestion is confirmed by a person first. The feature can also be disabled per-deployment if the bank prefers to launch without it.

Security & operations

VPC Service Controls + Cloud Armor / IAP

What: A network perimeter around the project and protection in front of the app.

Why: Prevents data from being exfiltrated outside the defined boundary and shields the application from common web attacks.

Secret Manager

What: Secure storage for API keys and service-account credentials.

Why: No secrets sit in plaintext config files; access is itself logged and controlled.

Cloud Audit Logs + Cloud Logging

What: A complete record of administrative actions and data access — who read or changed what, and when.

Why: Essential for a bank: supports compliance, internal investigation, and incident response. Combined with the system's own append-only history of goal/status changes, it gives a full audit trail.

Cloud Monitoring

What: Dashboards, alerting, and uptime checks.

Why: Operational visibility and the basis for a committed availability SLA.

Why this stack, in one breath

Every component is a **first-party Google Cloud service**. The app (Cloud Run), the data (Firestore), identity (Workspace SSO), the AI (Vertex), secrets, encryption, networking, and audit logging all live in **one tenant, under one security perimeter, on one DPA**. There is no exotic technology and no

rewrite required to get there — the existing application simply moves home and gets locked down. That is the shortest path to "MVP, bank-safe, entirely in your Google cloud."